OAVS TGT Computer Science

MCQ Question Bank

Preprocessor Directives in C

Topic 9 of OAVS Computer Teacher Examination Series

■ **Total MCQs**	135 Questions
■ **Topics Covered**	#include, #define, #ifdef, #ifndef, #undef, #error
■ **Includes**	Compiler macros, header files, conditional compilation
■ **Mock Tests**	2 Full Mock Tests (20 Qs each) with Answer Keys
■ **Exam**	OAVS Computer Teacher (TGT) Recruitment
■ **Difficulty**	Easy → Medium → Advanced (Progressive)

■■ *Note: Questions are categorised by directive/topic and progress from foundational to advanced. Each question includes a detailed explanation. Study all sections for complete exam preparedness.*

About This Book

What are Preprocessor Directives?

Preprocessor directives are commands processed by the C preprocessor before the actual compilation begins. They start with a # symbol and control the compilation environment. Key directives include #include (for file inclusion), #define (for macros), #ifdef/#ifndef (for conditional compilation), #undef (to remove macro definitions), and #error (to generate compile-time errors).

Why This Topic Matters for OAVS TGT Exam

The OAVS Computer Teacher exam regularly tests preprocessing concepts as they are fundamental to C programming. Questions range from simple syntax recognition to advanced macro expansion tracing and conditional compilation scenarios.

How to Use This Book

Work through sections sequentially. Each MCQ includes an explanation — read it even when you answer correctly to reinforce understanding. Attempt both mock tests under timed conditions (30 minutes each) to simulate exam pressure.

Exam Strategy Tips

- Always trace macro expansions step by step on paper.
- Remember: macros are TEXT substitution — no type checking, no scope rules.
- Include guards always use #ifndef (not #ifdef).
- #error stops compilation; #warning (GCC) does not.
- Predefined macros: __LINE__, __FILE__, __DATE__, __TIME__, __STDC__.

Quick Reference: Preprocessor Directives

Directive	Purpose	Example
#include	Include a header/source file	#include <stdio.h>
#define	Define a macro or constant	#define MAX 100
#undef	Remove a macro definition	#undef MAX
#ifdef	Compile if macro IS defined	#ifdef DEBUG
#ifndef	Compile if macro NOT defined	#ifndef MYHEADER_H
#if / #elif	Evaluate constant expression	#if VERSION >= 2
#else	Alternative conditional block	#else
#endif	End conditional block	#endif
#error	Force compile error with message	#error Missing config
#pragma	Implementation-specific directive	#pragma once
#line	Reset line number and filename	#line 100 "file.c"

Section 1: The #include Directive

1. Which preprocessor directive is used to include a header file in a C program?

A) #import
B) #include
C) #define
D) #header

Correct Answer: B)

Explanation: The #include directive tells the preprocessor to insert the contents of another file into the source code at that point. It is used to include standard or user-defined header files.

---

2. What is the correct syntax to include the standard input/output header in C?

A) #include (stdio.h)
B) #include stdio.h
C) #include <stdio.h>
D) include <stdio.h>

Correct Answer: C)

Explanation: #include <stdio.h> uses angle brackets to include system/standard header files. The angle brackets tell the preprocessor to search in the system include directories.

---

3. Which syntax is used to include a user-defined header file?

A) #include <myfile.h>
B) #include "myfile.h"
C) #include myfile
D) #include {myfile.h}

Correct Answer: B)

Explanation: Double quotes ("myfile.h") tell the preprocessor to first search in the current directory, then in standard include directories. Angle brackets search only in system directories.

---

4. The #include directive is processed by which component of the compiler?

A) Linker
B) Assembler
C) Preprocessor
D) Loader

Correct Answer: C)

Explanation: The C preprocessor (cpp) handles all directives beginning with #, including #include, before the actual compilation begins.

---

5. Which header file must be included to use printf() and scanf() in C?

A) stdlib.h
B) string.h
C) math.h
D) stdio.h

Correct Answer: D)

Explanation: stdio.h (Standard Input Output) provides declarations for printf(), scanf(), FILE, and other I/O functions. It must be included for standard I/O operations.

---

6. What happens when the preprocessor encounters #include <stdio.h>?

A) It compiles stdio.h separately
B) It replaces the directive with the content of stdio.h
C) It links the stdio library
D) It creates a copy of stdio.h

Correct Answer: B)

Explanation: The preprocessor literally replaces the #include directive with the entire contents of the specified header file before compilation proceeds.

---

7. Which header file is required to use functions like malloc() and free()?

A) stdio.h
B) string.h
C) stdlib.h
D) memory.h

Correct Answer: C)

Explanation: stdlib.h (Standard Library) provides declarations for dynamic memory allocation functions: malloc(), calloc(), realloc(), and free().

---

8. What is the difference between #include <file> and #include "file"?

A) No difference
B) <file> is for C++, "file" is for C
C) <file> searches system directories; "file" searches current directory first
D) <file> includes binary; "file" includes text

Correct Answer: C)

Explanation: Angle brackets instruct the preprocessor to search in predefined system include paths. Double quotes first check the current/project directory, then fall back to system paths.

---

9. Which header provides the strcpy() and strlen() functions?

A) stdio.h
B) string.h
C) stdlib.h
D) ctype.h

Correct Answer: B)

Explanation: string.h provides string handling functions: strcpy(), strcat(), strlen(), strcmp(), strstr(), and many more.

---

10. What is a header file in C?

A) A file with compiled object code
B) A file with macro definitions, function prototypes, and type declarations
C) The first file in a project
D) A file processed by the linker

Correct Answer: B)

Explanation: Header files (.h) typically contain macro definitions, type definitions, function prototypes, and global variable declarations that can be shared across multiple source files.

---

11. Which header must be included to use sqrt() in C?

A) stdlib.h
B) stdio.h
C) math.h
D) float.h

Correct Answer: C)

Explanation: math.h provides mathematical function declarations: sqrt(), pow(), sin(), cos(), log(), ceil(), floor(), and others. The math library (-lm) may also need to be linked.

---

12. Nested inclusion (a header including another header) is:

A) Not allowed in C
B) Allowed and very common
C) Allowed only in C99
D) Causes a compilation error

Correct Answer: B)

Explanation: C allows headers to include other headers. For example, many system headers internally include other headers. Include guards are used to prevent multiple inclusions of the same file.

---

13. What is an include guard used for?

A) To protect files from being deleted
B) To prevent a header file from being included more than once
C) To hide implementation details
D) To encrypt the header file

Correct Answer: B)

Explanation: Include guards use #ifndef, #define, and #endif to ensure that even if a header is included multiple times, its contents are only processed once, preventing duplicate declarations.

---

14. A correct include guard pattern looks like:

A) #if MYHEADER_H ... #end
B) #ifndef MYHEADER_H\n#define MYHEADER_H\n...\n#endif
C) #ifdef MYHEADER_H\n#define MYHEADER_H\n...\n#endif
D) #include_once MYHEADER_H

Correct Answer: B)

Explanation: The standard include guard: #ifndef checks if the macro is not defined, #define defines it, the file contents follow, and #endif closes the guard. This ensures single inclusion.

---

15. Which header is needed for isdigit() and isalpha() functions?

A) string.h
B) stdio.h
C) ctype.h
D) stdlib.h

Correct Answer: C)

Explanation: ctype.h provides character classification and conversion functions: isdigit(), isalpha(), islower(), isupper(), tolower(), toupper(), etc.

Section 2: The #define Directive & Macros

16. The #define directive is used to:

A) Declare a variable
B) Define a function
C) Create a macro or symbolic constant
D) Include a file

Correct Answer: C)

Explanation: #define creates macros — symbolic names replaced by the preprocessor before compilation. They can define constants, function-like macros, or any text substitution.

---

17. What does the following define? #define MAX 100

A) Declares an integer variable MAX with value 100
B) Creates a macro MAX that is text-replaced with 100 before compilation
C) Allocates memory of size 100
D) Defines a function named MAX

Correct Answer: B)

Explanation: Macros defined with #define are simple text substitutions. Every occurrence of MAX in the code will be replaced by 100 by the preprocessor before the compiler sees the code.

---

18. What is the output of: #define SQ(x) x*x

```
printf("%d", SQ(3));
```

A) 9
B) 6
C) 3
D) Error

Correct Answer: A)

*Explanation: SQ(3) expands to 3*3 = 9. However, this macro has a common pitfall: SQ(2+1) expands to 2+1*2+1 = 5, not 9. Always use parentheses in macros.*

---

19. Which of the following is the correct way to define PI as 3.14159?

A) #define PI = 3.14159
B) #define PI 3.14159
C) define PI 3.14159
D) #define PI: 3.14159

Correct Answer: B)

Explanation: #define syntax is: #define IDENTIFIER replacement_text (no = sign, no semicolon). The macro name is followed by a space and then the replacement text.

---

20. What is a function-like macro in C?

A) A macro that returns a value like a function
B) A macro that takes parameters and performs text substitution
C) A macro defined inside a function
D) A macro that calls another function

Correct Answer: B)

Explanation: Function-like macros take parameters: #define ADD(a,b) ((a)+(b)). They perform text substitution with their arguments but are not actual function calls — no type checking or stack frame.

---

21. What is wrong with this macro: #define SQUARE(x) x*x ?

A) Nothing is wrong
B) Missing semicolon
C) No parentheses around x, causing incorrect expansion for expressions
D) #define cannot define functions

Correct Answer: C)

*Explanation: SQUARE(2+3) expands to 2+3*2+3 = 13, not 25. The correct definition should be #define SQUARE(x) ((x)*(x)) with parentheses around each parameter and the whole expression.*

---

22. Can macros be defined with multiple lines?

A) No, macros must be on one line
B) Yes, using \ at the end of each line to continue
C) Yes, using { } to enclose lines
D) Yes, but only in C99

Correct Answer: B)

Explanation: Macros can span multiple lines using the line-continuation character \. The preprocessor treats all continued lines as a single logical line.

---

23. What is the purpose of parentheses in macro definitions like #define ADD(a,b) ((a)+(b))?

A) They are mandatory syntax for macros
B) They prevent operator precedence issues during macro expansion
C) They make the macro faster
D) They allow the macro to return a value

Correct Answer: B)

Explanation: Parentheses around each parameter and the whole expression prevent unexpected operator precedence issues when complex expressions are passed as macro arguments.

---

24. #define without a replacement value (e.g., #define DEBUG) is used for:

A) Defining an empty string
B) Conditional compilation with #ifdef/#ifndef
C) This is a syntax error
D) Creating a null pointer

Correct Answer: B)

Explanation: A macro defined without a value simply marks that identifier as defined. This is used with #ifdef to conditionally compile sections of code, common for debug builds.

---

25. Which of the following correctly defines a macro for the maximum of two numbers?

A) #define MAX(a,b) a>b?a:b
B) #define MAX(a,b) ((a)>(b)?(a):(b))
C) #define MAX(a,b) = ((a)>(b)?(a):(b))
D) #macro MAX(a,b) ((a)>(b)?(a):(b))

Correct Answer: B)

Explanation: Option B is correct with full parenthesization. Option A lacks parentheses and can cause wrong results with complex expressions. The keyword is #define, not #macro.

---

26. What does the predefined macro __LINE__ represent?

A) The length of the current line
B) The current line number in the source file
C) The line of the last error

D) The number of lines in the file

Correct Answer: B)

Explanation: __LINE__ is a predefined macro that expands to the current line number (an integer) in the source file. It's useful for debugging and error messages.

---

27. What does the predefined macro __FILE__ represent?

A) The size of the file
B) The name of the current source file as a string literal
C) A file pointer
D) The number of files included

Correct Answer: B)

Explanation: __FILE__ expands to the name of the current source file as a string literal. Combined with __LINE__, it helps produce informative debug/assert messages.

---

28. What does __DATE__ macro expand to?

A) The current runtime date
B) The compilation date as a string (e.g., "Jan 01 2024")
C) The date the file was last modified
D) A timestamp integer

Correct Answer: B)

Explanation: __DATE__ is a predefined macro that expands to the compilation date as a string literal in the form "Mmm dd yyyy". It is fixed at compile time, not runtime.

---

29. Macros are processed at which stage?

A) Runtime
B) Link time
C) Compile time (preprocessing phase)
D) Load time

Correct Answer: C)

Explanation: Macros are processed by the C preprocessor before compilation. The preprocessor performs text substitution and produces modified source code that the compiler then processes.

---

30. What is the advantage of using #define constants over regular variables?

A) Macros are stored in memory
B) Macros have type checking
C) Macros have no memory overhead and are resolved at compile time
D) Macros are faster at runtime

Correct Answer: C)

Explanation: #define constants are substituted at preprocessing time — they don't occupy memory at runtime. Variables require memory allocation. However, macros have no type safety, which is a disadvantage.

---

31. What is the output? #define A 2+3

```
printf("%d", A*A);
```

A) 25
B) 11
C) 10
D) Compilation error

Correct Answer: B)

*Explanation: A*A expands to 2+3*2+3. Due to operator precedence, multiplication is done first: 2 + (3*2) + 3 = 2+6+3 = 11. This illustrates the importance of parentheses in macros.*

32. Which predefined macro gives the time of compilation?

A) __DATE__
B) __TIME__
C) __CLOCK__
D) __TIMESTAMP__

Correct Answer: B)

Explanation: __TIME__ expands to the time of compilation as a string literal in the form "HH:MM:SS". Like __DATE__, it is fixed at compile time.

33. Can a macro name be the same as a variable name?

A) Yes, and the macro takes precedence
B) No, it causes a compile error
C) Yes, but the variable takes precedence
D) Only in C++

Correct Answer: A)

Explanation: If a macro has the same name as a variable, the preprocessor replaces all occurrences of that name with the macro's expansion before the compiler sees the code, so the macro takes precedence.

34. What does #define NULL 0 typically represent?

A) An empty string
B) A null pointer constant
C) Zero as an integer only
D) An undefined value

Correct Answer: B)

Explanation: NULL is traditionally defined as 0 (or (void)0 in C) in header files like stdio.h and stdlib.h. It represents a null pointer constant used to indicate that a pointer points to nothing.*

Section 3: #ifdef and #ifndef Directives

35. What does #ifdef stand for?

A) if defined — checks if a macro is defined
B) if block defined
C) integer function defined
D) include file defined

Correct Answer: A)

Explanation: #ifdef (if defined) is a conditional compilation directive. The code block following #ifdef MACRO is compiled only if MACRO has been defined using #define.

---

36. What is the output if DEBUG is not defined?

```
#ifdef DEBUG
printf("Debug mode");
#else
printf("Release");
#endif
```

A) Debug mode
B) Release
C) Nothing
D) Compilation error

Correct Answer: B)

Explanation: Since DEBUG is not defined, #ifdef DEBUG is false. The #else branch executes, printing "Release". This pattern is standard for toggling debug code.

---

37. What does #ifndef mean?

A) if not then
B) if not defined — true when macro is NOT defined
C) include not template
D) integer not found

Correct Answer: B)

Explanation: #ifndef (if not defined) is the opposite of #ifdef. The code block is compiled only if the specified macro has NOT been defined. It's the basis of include guards.

---

38. Which directive is used to end a conditional compilation block?

A) #end
B) #close
C) #endif
D) #stop

Correct Answer: C)

Explanation: #endif terminates a conditional compilation block started by #if, #ifdef, or #ifndef. Every conditional directive must be paired with #endif.

---

39. What is the primary use of #ifndef in header files?

A) To check if a function exists
B) To prevent multiple inclusion of the same header file (include guards)
C) To define new types

D) To import modules

Correct Answer: B)

Explanation: #ifndef HEADER_H / #define HEADER_H / ... / #endif is the standard include guard pattern. If the header is included again, HEADER_H is already defined so #ifndef is false and the content is skipped.

---

40. What does the following code compile?

```
#define WINDOWS
#ifdef WINDOWS
printf("Win");
#endif
```

A) Nothing (error)
B) printf("Win");
C) #define WINDOWS line
D) Skips all code

Correct Answer: B)

Explanation: Since WINDOWS is defined (even without a value), #ifdef WINDOWS is true. The printf statement is included in compilation and will print "Win".

---

41. Can #ifdef and #ifndef be nested?

A) No
B) Yes, but only one level
C) Yes, multiple levels of nesting are allowed
D) Only in C99 and later

Correct Answer: C)

Explanation: Conditional compilation directives can be nested to any depth. Each #ifdef/#ifndef must have its own matching #endif. This allows complex conditional compilation scenarios.

---

42. What is the difference between #ifdef and #if defined()?

A) No difference in functionality
B) #if defined() allows logical operators like &&, ||
C) #ifdef is faster
D) #if defined() is deprecated

Correct Answer: B)

Explanation: #if defined(MACRO) is equivalent to #ifdef MACRO for a single check. But #if defined() allows combining: #if defined(A) && defined(B) — this is not possible with #ifdef directly.

---

43. What happens if #ifdef has no matching #endif?

A) The compiler ignores it
B) Compilation error: unterminated conditional
C) The rest of the file is excluded
D) A warning is generated but compilation continues

Correct Answer: B)

Explanation: Every #ifdef, #ifndef, or #if must have a matching #endif. If one is missing, the compiler/preprocessor reports an error for an unterminated conditional directive.

---

44. How many #else branches can a single #ifdef have?

A) Unlimited
B) Two
C) Only one

D) None

Correct Answer: C)

Explanation: Each #ifdef/#ifndef block can have at most one #else branch. However, multiple #elif branches are allowed. The structure is: #ifdef ... [#elif ...] [#else ...] #endif.

---

45. What does #elif stand for?

A) else include

B) else if — allows chaining conditional compilation checks

C) end link

D) external library

Correct Answer: B)

Explanation: #elif (else if) allows chaining multiple conditions in conditional compilation, similar to else-if in regular C code. It avoids deeply nested #ifdef/#endif structures.

---

46. Consider: #ifndef PI

```
#define PI 3.14
#endif. What is the purpose?
```

A) Defines PI only if it hasn't been defined already

B) Defines PI always

C) Undefines PI

D) Checks if PI equals 3.14

Correct Answer: A)

Explanation: This pattern safely defines PI only if it's not already defined. If a user has defined their own PI elsewhere, this won't override it. It's a defensive definition pattern.

---

47. Which directive is used to compile code for Windows platform specifically?

A) #include <windows.h>

B) #ifdef _WIN32 ... #endif

C) #define WINDOWS

D) #if OS == WINDOWS

Correct Answer: B)

Explanation: _WIN32 is automatically defined by Microsoft compilers on Windows. Using #ifdef _WIN32 allows writing platform-specific code that only compiles on Windows.

---

48. What is the output?

```
#define X
#ifndef X
printf("A");
#else
printf("B");
#endif
```

A) A

B) B

C) AB

D) Nothing

Correct Answer: B)

Explanation: X is defined, so #ifndef X is false (X IS defined). The #else branch executes, printing "B".

---

49. Which of the following represents a correct include guard?

A) #ifdef MYLIB_H\n#define MYLIB_H\n#endif
B) #ifndef MYLIB_H\n#define MYLIB_H\n/* contents */\n#endif
C) #if MYLIB_H\n#define MYLIB_H\n#endif
D) #once MYLIB_H

Correct Answer: B)

Explanation: The correct include guard uses #ifndef (not #ifdef). It checks if the macro is NOT yet defined, defines it, includes the file contents, then closes with #endif.

---

50. The #if directive can evaluate:

A) Only defined/undefined macros
B) Only integer constant expressions
C) Any C expression including function calls
D) String comparisons

Correct Answer: B)

Explanation: #if evaluates constant integer expressions at preprocessing time. It cannot evaluate runtime values, string comparisons, or function calls (except the special defined() operator).

Section 4: The #undef Directive

51. What is the purpose of the #undef directive?

A) To declare undefined behavior
B) To remove/undefine a previously defined macro
C) To undefine a variable
D) To undo #include

Correct Answer: B)

Explanation: #undef removes a macro definition. After #undef MACRO_NAME, the macro no longer exists and #ifdef MACRO_NAME would be false. This allows redefining macros with different values.

52. What is the result of using #undef on an undefined macro?

A) Compilation error
B) Runtime error
C) No effect (it is silently ignored)
D) Warning and termination

Correct Answer: C)

Explanation: If you #undef a macro that was never defined or was already undefined, the preprocessor silently ignores it. No error or warning is generated.

53. When would you use #undef followed by #define?

A) To change the value of a macro in a specific code section
B) To permanently remove a macro
C) To create a new variable
D) To reset all macros

Correct Answer: A)

Explanation: #undef removes the current definition, then #define creates a new one. This allows temporarily redefining a macro with a different value for a specific portion of code.

54. What does this code do?

```
#define SIZE 10
/* ... */
#undef SIZE
#define SIZE 20
```

A) Error: SIZE already defined
B) SIZE is 10 before #undef, then redefined as 20 after
C) SIZE is always 20
D) SIZE is always 10

Correct Answer: B)

Explanation: Code before the #undef uses SIZE = 10. After #undef, SIZE is no longer defined. #define SIZE 20 then creates a fresh definition. Code after this point uses SIZE = 20.

55. Can you #undef a predefined macro like __LINE__?

A) Yes, and it works normally
B) It is undefined behavior
C) No, predefined macros cannot be undefined
D) It causes a compile error in all compilers

Correct Answer: B)

Explanation: While some compilers may allow it syntactically, undefining predefined macros like __LINE__ or __FILE__ results in undefined behavior according to the C standard. It should be avoided.

---

56. After #undef MAX, what does #ifdef MAX evaluate to?

A) True
B) False — MAX is no longer defined
C) Error
D) True if MAX was defined before

Correct Answer: B)

Explanation: After #undef MAX, the macro MAX does not exist. #ifdef MAX checks if MAX is currently defined, which it is not, so it evaluates to false.

---

57. What is the typical use case for #undef in library code?

A) To delete functions from the program
B) To clean up helper macros that should not be visible to users of the library
C) To reset the compiler
D) To remove #include directives

Correct Answer: B)

Explanation: Library code often defines helper macros internally. Using #undef at the end of the header prevents these implementation-detail macros from polluting the user's namespace.

---

58. Redefining a macro WITHOUT #undef typically causes:

A) Compilation error
B) The new definition silently replaces the old one
C) A warning from the compiler
D) Undefined behavior

Correct Answer: C)

Explanation: Most compilers issue a warning when you redefine a macro without first using #undef. It is not an error, but the warning alerts the programmer to a potential unintentional redefinition.

---

59. Which of the following is the correct syntax for #undef?

A) #undef(MACRO_NAME)
B) #undef MACRO_NAME
C) undef MACRO_NAME;
D) #undefine MACRO_NAME

Correct Answer: B)

Explanation: The correct syntax is simply #undef followed by the macro name. No parentheses, no semicolon. It is: #undef IDENTIFIER

---

60. What is the scope of a macro defined with #define?

A) Only within the function it is defined in
B) From the point of definition to the end of the file (or until #undef)
C) Only within the block {} it is defined in
D) Global across all files always

Correct Answer: B)

Explanation: Macros have file scope — they are valid from the point of #define to the end of the translation unit, or until #undef is encountered. They do not have block scope like C variables.

Section 5: The #error Directive

61. What is the purpose of the #error directive?

A) To handle runtime errors
B) To generate a compilation error with a custom message
C) To define error codes
D) To catch exceptions

Correct Answer: B)

Explanation: #error immediately stops compilation and displays a user-defined error message. It is used to enforce requirements that must be met for the code to compile correctly.

62. What is the correct syntax of the #error directive?

A) #error("message")
B) #error: message
C) #error message
D) error #"message"

Correct Answer: C)

Explanation: The syntax is #error followed by the error message text (not in quotes). The preprocessor outputs this message and halts compilation.

63. When is #error typically used?

A) To debug runtime issues
B) To ensure certain macros are defined or platform requirements are met before compiling
C) To throw C exceptions
D) To print to stderr at runtime

Correct Answer: B)

Explanation: #error is commonly used in conditional compilation: if a required configuration macro is missing, #error forces a meaningful compile-time failure rather than a cryptic compiler error later.

64. What happens when the preprocessor encounters #error Unsupported platform?

A) It prints "Unsupported platform" at runtime
B) It stops compilation and shows "Unsupported platform" as an error
C) It skips the remaining code
D) It generates a warning only

Correct Answer: B)

Explanation: #error immediately halts preprocessing/compilation and displays the given message as a compiler error. The program cannot be built, which forces the developer to address the issue.

65. Which of the following uses #error correctly to enforce platform?

A) #if !defined(__linux__)\n#error Linux required\n#endif
B) #error if !linux
C) if (!linux) #error
D) #define error Linux required

Correct Answer: A)

Explanation: Option A checks if the __linux__ macro is not defined (i.e., not on Linux) and generates a compile error with the message "Linux required". This is the standard pattern for platform enforcement.

66. Can #error appear inside a #ifdef block?

A) No
B) Yes — it only triggers if the enclosing condition is true
C) Only in C11
D) Only as the first directive in a file

Correct Answer: B)

Explanation: #error inside a conditional block only executes if that block's condition is true. This allows conditional error messages, such as: #ifdef WRONG_CONFIG\n#error Bad config\n#endif

---

67. Which directive can generate a non-fatal warning during compilation?

A) #error
B) #warning
C) #note
D) #alert

Correct Answer: B)

Explanation: #warning (a common compiler extension, supported by GCC/Clang) generates a warning message without stopping compilation. Unlike #error, compilation continues after a #warning.

---

68. What is the difference between #error and assert()?

A) No difference
B) #error is a compile-time directive; assert() is a runtime check
C) assert() is faster
D) #error is only for C++

Correct Answer: B)

Explanation: #error triggers at preprocessing/compile time and prevents the program from being built. assert() is a runtime macro that checks a condition and terminates the program if it's false.

---

69. What does this code do?

```
#ifndef __cplusplus
#error C++ compiler required
#endif
```

A) Requires a C compiler
B) Generates an error if not compiled with a C++ compiler
C) Generates an error always
D) Does nothing

Correct Answer: B)

Explanation: __cplusplus is defined by C++ compilers. If it's not defined (i.e., a C compiler is being used), #error fires. This ensures the file is compiled only as C++.

---

70. The #error directive is evaluated by:

A) The runtime system
B) The linker
C) The preprocessor
D) The assembler

Correct Answer: C)

Explanation: Like all # directives, #error is processed by the C preprocessor before compilation. If reached, it reports the error and stops further processing.

Section 6: General Preprocessor Concepts

71. What is the C preprocessor?

A) A program that links object files
B) A program that performs text transformations on source code before compilation
C) The compiler's optimizer
D) A runtime library

Correct Answer: B)

Explanation: The C preprocessor (cpp) is a separate pass that runs before the compiler. It handles all # directives: including files, expanding macros, conditional compilation, and line control.

72. Which of the following is NOT a preprocessor directive?

A) #include
B) #define
C) #while
D) #ifdef

Correct Answer: C)

Explanation: There is no #while preprocessor directive in C. The valid conditional directives are #if, #ifdef, #ifndef, #elif, #else, and #endif. #while is a regular C statement keyword.

73. Preprocessor directives in C always begin with:

A) //
B) @
C) #
D) $

Correct Answer: C)

Explanation: All preprocessor directives start with the # symbol (hash/pound). They must appear at the beginning of a line (optionally preceded by whitespace in C99 and later).

74. Which of the following represents a comment in C that the preprocessor ignores?

A) <!-- comment -->
B) /* comment */
C) # comment
D) // comment

Correct Answer: B)

Explanation: C-style comments (/ */) and C++ style (//) are removed before preprocessing. The preprocessor sees the source without comments. Note: # is a preprocessor directive marker, not a comment.*

75. What is the translation phase order in C compilation?

A) Compilation → Preprocessing → Linking
B) Preprocessing → Compilation → Assembly → Linking
C) Linking → Compilation → Preprocessing
D) Assembly → Preprocessing → Compilation

Correct Answer: B)

Explanation: The standard order is: Preprocessing (handle # directives) → Compilation (source to assembly) → Assembly (assembly to object code) → Linking (combine object files into executable).

76. What does #pragma once do?

A) Runs the program once
B) A non-standard but widely supported directive ensuring a header is included only once
C) Defines a one-time macro
D) Compiles a file once per build

Correct Answer: B)

Explanation: #pragma once is a widely used alternative to include guards. It tells the preprocessor to include the file only once per compilation unit. It's not in the C standard but supported by most modern compilers.

---

77. What is a translation unit in C?

A) A compiled binary
B) A single source file after preprocessing (including all #included content)
C) One function in a source file
D) A library module

Correct Answer: B)

Explanation: A translation unit is a source file (.c) after the preprocessor has processed it — expanding macros and inserting #included headers. It is the basic unit of compilation.

---

78. Macro arguments in function-like macros are:

A) Type-checked by the compiler
B) Evaluated before substitution always
C) Substituted as text without type checking
D) Passed by reference

Correct Answer: C)

Explanation: Macro arguments are textually substituted into the macro body without any type checking. This is both a strength (generic code) and a weakness (no safety). Inline functions are safer alternatives.

---

79. What is the stringification operator in C macros?

A) ##
B) #
C) @
D) $$

Correct Answer: B)

Explanation: The # operator in a macro definition converts a macro argument to a string literal. For example: #define STR(x) #x — STR(hello) becomes "hello".

---

80. What is the token-pasting operator in C macros?

A) #
B) ##
C) @@
D) ++

Correct Answer: B)

Explanation: The ## (token-paste or concatenation) operator joins two tokens together. For example: #define CONCAT(a,b) a##b — CONCAT(foo,bar) becomes foobar (a single identifier).

---

81. What does #define TRUE 1 establish?

A) A variable named TRUE equal to 1
B) A macro that replaces TRUE with 1 throughout the code
C) A constant of type bool
D) A function returning 1

Correct Answer: B)

Explanation: This defines a macro. Every occurrence of TRUE in the code after this definition will be replaced by 1 by the preprocessor. No memory is allocated.

---

82. Which header contains the definition of TRUE and FALSE in C99?

A) stdio.h
B) stdlib.h
C) stdbool.h
D) bool.h

Correct Answer: C)

Explanation: C99 introduced stdbool.h which defines the bool type, true (as 1), and false (as 0) as macros. It also defines __bool_true_false_are_defined.

---

83. What is the output?

```
#define DOUBLE(x) x+x
int a=3;
printf("%d",DOUBLE(a)*2);
```

A) 12
B) 9
C) 6
D) Error

Correct Answer: B)

*Explanation: DOUBLE(a)*2 expands to a+a*2 = 3+(3*2) = 3+6 = 9 due to precedence. DOUBLE should be defined as #define DOUBLE(x) ((x)+(x)) to give correct result of 12.*

---

84. Which of these macros is predefined in all standard C compilers?

A) __AUTHOR__
B) __VERSION__
C) __STDC__
D) __CPU__

Correct Answer: C)

Explanation: __STDC__ is defined as 1 if the compiler conforms to the C standard. Other predefined macros include __LINE__, __FILE__, __DATE__, __TIME__, and __STDC_VERSION__.

---

85. What does __STDC_VERSION__ indicate?

A) The version of GCC
B) The C standard version being used (e.g., 199901L for C99)
C) The number of standard headers
D) The version of the standard library

Correct Answer: B)

Explanation: __STDC_VERSION__ gives the C standard version: 199409L for C94, 199901L for C99, 201112L for C11, 201710L for C17. It allows writing code compatible with specific standard versions.

---

86. Can the same file be #included twice in the same source file?

A) No — compilation error
B) Yes, but without include guards the content is processed twice causing errors
C) Yes, duplicate includes are always ignored
D) Only in C++ but not C

Correct Answer: B)

Explanation: Without include guards, the same header included twice results in duplicate declarations, causing compilation errors. Include guards (#ifndef/#define/#endif) or #pragma once prevent this.

---

87. What is the role of the ## operator in this macro?

```
#define VAR(n) var##n
VAR(1) = 5;
```

A) Creates a string
B) Concatenates var and n to form the identifier var1
C) Multiplies n by 2
D) Defines an array

Correct Answer: B)

Explanation: ## is the token-pasting operator. VAR(1) expands to var##1 which becomes the single token var1. So VAR(1) = 5; is equivalent to var1 = 5;

---

88. The # operator in a macro is called:

A) Token pasting
B) Stringification/Stringizing
C) Concatenation
D) Dereferencing

Correct Answer: B)

Explanation: The unary # operator in a macro definition converts a macro parameter to a string literal (surrounded by double quotes). This is called stringification or stringizing.

---

89. Which of the following macro definitions allows computing the minimum of two values safely?

A) #define MIN(a,b) a<b?a:b
B) #define MIN(a,b) ((a)<(b)?(a):(b))
C) #define MIN(a,b) min(a,b)
D) #define MIN = (a<b?a:b)

Correct Answer: B)

Explanation: Option B uses full parenthesization — both around each parameter and the whole expression — making it safe for complex expressions. Option A can produce incorrect results with operators of lower precedence.

---

90. What does the following evaluate to?

```
#if 2 > 1
printf("Yes");
#else
printf("No");
#endif
```

A) No
B) Yes
C) Compilation error
D) Nothing

Correct Answer: B)

Explanation: #if evaluates constant integer expressions at compile time. 2 > 1 is true (non-zero), so the first branch is compiled and the program prints "Yes".

---

91. Macros are generally preferred over functions when:

A) Type safety is important

B) The overhead of a function call must be avoided and the code is small
C) The code is complex and long
D) Debug builds are needed

Correct Answer: B)

Explanation: Macros eliminate function call overhead (no stack frame, no parameter passing). They're suitable for small, frequently-used operations. However, inline functions in C99+ are usually preferred for type safety.

---

92. What is the maximum number of parameters a function-like macro can have?

A) 8
B) 16
C) 32
D) Implementation-defined (but at least 127 per C99)

Correct Answer: D)

Explanation: The C standard does not set a fixed limit on macro parameters, but requires implementations to support at least 127. In practice, very large numbers of parameters are rarely needed.

---

93. Variadic macros (macros with variable arguments) use which notation?

A) ...
B) *args
C) **
D) []

Correct Answer: A)

Explanation: C99 introduced variadic macros using ... (ellipsis) as the last parameter and __VA_ARGS__ in the replacement text. Example: #define LOG(fmt,...) printf(fmt, __VA_ARGS__)

---

94. What does #line directive do?

A) Prints the current line number
B) Changes the reported line number and optionally the file name for subsequent diagnostics
C) Skips the next line
D) Marks a line as important

Correct Answer: B)

Explanation: #line n "file" tells the preprocessor to report subsequent lines as starting from line n in file "file". Used by code generators to map back to the original source.

---

95. Which operator checks whether a macro is defined inside an #if expression?

A) exists()
B) isdefined()
C) defined()
D) check()

Correct Answer: C)

Explanation: The defined() operator is used within #if and #elif expressions: #if defined(MACRO) is equivalent to #ifdef MACRO. It allows complex logical combinations: #if defined(A) && !defined(B)

---

96. What is the output?

```
#define STR(s) #s
printf("%s", STR(Hello));
```

A) s
B) #Hello
C) Hello

D) "Hello" with quotes

Correct Answer: C)

Explanation: The # stringification operator converts the argument Hello into the string literal "Hello". printf("%s", "Hello") prints Hello (without quotes, as %s prints the string contents).

---

97. Which of the following headers provides INT_MAX and INT_MIN?

A) limits.h
B) stdio.h
C) stdlib.h
D) math.h

Correct Answer: A)

Explanation: limits.h defines macros for the ranges of integer types: INT_MAX, INT_MIN, LONG_MAX, CHAR_MAX, etc. These are implementation-defined constants for the target platform.

---

98. What does #pragma do in C?

A) Defines a standard preprocessor directive
B) Issues implementation-specific instructions to the compiler
C) Includes a binary file
D) Defines a new data type

Correct Answer: B)

Explanation: #pragma provides implementation-defined instructions. Common uses: #pragma once (include guard), #pragma pack (structure packing), #pragma warning (warning control). Unknown pragmas are typically ignored.

---

99. A macro defined inside a function:

A) Has local scope to the function
B) Is not allowed by the C standard
C) Has file scope from point of definition (macros ignore block scope)
D) Is automatically undefined at function end

Correct Answer: C)

Explanation: Macros don't respect block scope. A macro defined inside a function body is visible from that point to the end of the file (or until #undef). The concept of local macros doesn't exist.

---

100. What is the output?

```
#define CAT(a,b) a##b
int xy = 10;
printf("%d", CAT(x,y));
```

A) Error
B) xy
C) 10
D) 0

Correct Answer: C)

Explanation: CAT(x,y) uses token-pasting to form the identifier xy. Since int xy = 10 is declared, printf("%d", xy) prints 10.

Section 7: Advanced Macro Techniques

101. What is macro hygiene in C?

A) Keeping macro names clean and short
B) Using unique names in macro expansions to avoid variable capture
C) Deleting macros after use
D) Writing macros in uppercase only

Correct Answer: B)

Explanation: Macro hygiene refers to avoiding unintended interactions between macro-expanded names and names in the surrounding code (variable capture). C macros are not hygienic; this is a known limitation.

---

102. Which of the following avoids the double-evaluation problem in macros?

A) Using more parentheses
B) Using inline functions
C) Using #undef
D) Using #if

Correct Answer: B)

Explanation: Macros like #define MAX(a,b) ((a)>(b)?(a):(b)) evaluate a or b twice if expressions with side effects are passed (e.g., MAX(i++, j++)). Inline functions evaluate arguments exactly once, avoiding this problem.

---

103. What is the do...while(0) trick in multi-statement macros?

A) Makes the macro loop
B) Wraps multiple statements so the macro behaves like a single statement (works with if-else)
C) Prevents macro expansion
D) Defines a loop macro

Correct Answer: B)

Explanation: #define MULTI(x) do { stmt1; stmt2; } while(0) allows the macro to be used safely after if/else. Without do...while(0), if(x) MULTI(x); would only execute stmt1 under the if.

---

104. What is the output?

```
#define NUM 5
#undef NUM
#define NUM 10
printf("%d", NUM);
```

A) 5
B) 10
C) 15
D) Error

Correct Answer: B)

Explanation: NUM is first defined as 5, then undefined with #undef, then redefined as 10. The printf statement sees NUM = 10 and prints 10.

---

105. What does the following check?

```
#if defined(__GNUC__) && __GNUC__ >= 4
```

A) Whether GCC version is at most 4
B) Whether the compiler is GCC version 4 or higher
C) Whether GCC is not installed

D) Whether C standard is version 4

Correct Answer: B)

Explanation: defined(__GNUC__) checks if the compiler is GCC. __GNUC__ gives the major version number. The condition is true when using GCC 4.x or later. This allows GCC-specific feature usage.

---

106. Can macros be used to implement generic data structures in C?

A) No, macros cannot create types

B) Yes, by using token-pasting to generate type-specific code

C) Only with C11 _Generic

D) Only using function pointers

Correct Answer: B)

Explanation: Token-pasting (##) allows macros to generate type-specific identifiers: #define DEFINE_STACK(T) T T##_stack[100]; / etc */ — enabling generic containers in C without templates.*

---

107. What is X-macros technique in C?

A) Macros starting with X

B) A pattern using #define with a macro argument to generate repetitive code from a list

C) Macros for cross-platform code

D) A deprecated feature of old C

Correct Answer: B)

Explanation: X-macros define a list in terms of a macro X that is defined differently each time the list is expanded. This keeps enum values, array initializers, and string arrays in sync automatically.

---

108. Which of these is a valid use of #error for version checking?

A) #error if VERSION < 2

B) #if VERSION < 2\n#error Version 2 or higher required\n#endif

C) #define #error VERSION

D) #include <error.h>

Correct Answer: B)

Explanation: Option B correctly uses #if to check a version macro and #error to halt compilation with an informative message if the requirement is not met.

---

109. What is the purpose of the __attribute__ keyword in GCC?

A) A standard C preprocessor directive

B) A GCC extension for specifying attributes for functions, variables, and types

C) A C99 macro

D) An alias for #pragma

Correct Answer: B)

Explanation: __attribute__ is a GCC-specific extension (not standard C) used to specify properties like __attribute__((deprecated)), __attribute__((noreturn)), and __attribute__((packed)). It's often wrapped in macros for portability.

---

110. Which preprocessor directive is used to display a custom message without stopping compilation in GCC?

A) #error

B) #message

C) #warning

D) #note

Correct Answer: C)

Explanation: #warning is a GCC/Clang extension that emits a warning message during compilation but does NOT stop compilation (unlike #error). It's useful for marking deprecated features or reminding about TODO items.

---

111. What happens when you define a macro that conflicts with a function name?

A) Compilation error always
B) The macro takes precedence in preprocessing; function calls may be replaced
C) The function takes precedence
D) Both coexist with no conflict

Correct Answer: B)

Explanation: Preprocessor substitution happens before the compiler sees the code. If a macro has the same name as a function, all uses of that name will be macro-expanded. This can hide the function.

---

112. What does #define _POSIX_C_SOURCE 200809L typically do?

A) Defines the POSIX version for feature-test macros
B) Sets the C standard version
C) Enables POSIX system calls at runtime
D) Links the POSIX library

Correct Answer: A)

Explanation: Feature-test macros like _POSIX_C_SOURCE control which POSIX API features are exposed by system headers. Setting it to 200809L enables POSIX.1-2008 features in the program's headers.

---

113. In which section of a C file should #include directives typically appear?

A) Inside main()
B) Inside function bodies only
C) At the top of the file before any declarations
D) After all function definitions

Correct Answer: C)

Explanation: By convention, #include directives appear at the top of source files before any declarations or code. While technically possible elsewhere, top placement is the universal standard.

---

114. What is the effect of #define with an empty replacement?

```
#define EMPTY
```

A) EMPTY is replaced with 0
B) EMPTY is replaced with an empty string (nothing)
C) Compilation error
D) EMPTY is treated as NULL

Correct Answer: B)

Explanation: When #define EMPTY has no replacement text, every occurrence of EMPTY in the code is replaced with nothing (empty). This is used to mark macros as defined for #ifdef testing.

---

115. What is the output?

```
#define A(x) #x
printf(A(world));
```

A) world
B) x
C) A(world)
D) "world"

Correct Answer: A)

Explanation: A(world) expands to #world which stringifies to "world". printf("%s", "world") ... but since %s isn't specified, the format string itself is "world" and it prints world.

---

116. Which of the following correctly uses conditional compilation to handle 32-bit vs 64-bit?

A) #if sizeof(int)==4 ... #endif
B) #if defined(__LP64__) ... #else ... #endif
C) #if runtime_bits==64 ... #endif
D) #ifdef 64 ... #endif

Correct Answer: B)

Explanation: Sizeof cannot be used in #if (it's not a constant expression at preprocessing time). Platform macros like __LP64__, _WIN64, or __x86_64__ are used in #ifdef/#if defined() for 32/64-bit selection.

---

117. What does this define? #define ARRAY_SIZE(arr) (sizeof(arr)/sizeof((arr)[0]))

A) The size in bytes of arr
B) The number of elements in an array arr
C) A pointer to arr
D) The maximum index of arr

Correct Answer: B)

Explanation: sizeof(arr) gives total bytes; sizeof(arr[0]) gives bytes per element. Dividing gives the count. This is a standard C macro for array size. Note: it fails for pointer arguments.

---

118. Which header file is included for errno and error codes?

A) error.h
B) err.h
C) errno.h
D) stdio.h

Correct Answer: C)

Explanation: errno.h provides the errno macro (an integer representing the last error code) and error code constants like ENOENT, ENOMEM, EACCES, etc., used by system and library functions.

---

119. What is the result of multiple #define for the same macro WITHOUT #undef between them?

A) The last definition wins
B) Compilation error
C) Compiler warning; behavior implementation-defined for which value is used
D) Both definitions are merged

Correct Answer: C)

Explanation: According to the C standard, redefining a macro that is already defined (with a different replacement) requires an #undef first; otherwise it is a constraint violation. Most compilers issue a warning.

---

120. Which of the following is TRUE about inline functions vs macros?

A) Macros are always faster
B) Inline functions provide type checking; macros do not
C) Macros can be debugged easily
D) Inline functions cannot be recursive

Correct Answer: B)

Explanation: Inline functions are processed by the compiler, providing full type checking, proper scoping, and debuggability. Macros are preprocessor text substitutions with no type safety. C99 inline functions are generally preferred.

Section 8: Header Files Deep Dive

121. What does the assert() macro do, and where is it defined?

A) Allocates memory; defined in stdlib.h
B) Checks a condition; terminates if false; defined in assert.h
C) Defines error codes; defined in errno.h
D) Prints a message; defined in stdio.h

Correct Answer: B)

Explanation: assert(expr) evaluates expr; if false (0), it prints a diagnostic message to stderr and calls abort(). Defining NDEBUG before including assert.h disables all assertions.

122. What does NDEBUG macro do when defined before #include <assert.h>?

A) Enables debug mode
B) Disables assert() — all assertions become no-ops
C) Defines the DEBUG macro
D) Enables verbose output

Correct Answer: B)

Explanation: When NDEBUG is defined, the assert.h header defines assert(expr) as ((void)0) — a no-op. This allows removing all assertion overhead in release builds without changing code.

123. The header <stddef.h> defines which common macro?

A) printf
B) NULL and size_t and ptrdiff_t
C) INT_MAX
D) bool

Correct Answer: B)

Explanation: stddef.h defines fundamental types and macros: NULL (null pointer constant), size_t (unsigned type for sizes), ptrdiff_t (type for pointer differences), and offsetof() macro.

124. Which macro gives the byte offset of a member within a struct?

A) sizeof()
B) memberof()
C) offsetof()
D) addressof()

Correct Answer: C)

Explanation: offsetof(type, member) defined in stddef.h returns the byte offset of a struct member from the beginning of the struct. Used in serialization, memory layout analysis, and container_of patterns.

125. What is the purpose of #include <float.h>?

A) To enable floating point math
B) To provide macros defining limits of floating-point types (FLT_MAX, DBL_MIN, etc.)
C) To include the math library
D) To define float as a type

Correct Answer: B)

Explanation: float.h defines implementation limits for floating-point types: FLT_MAX (max float), FLT_MIN (min positive float), DBL_MAX, DBL_EPSILON, FLT_DIG (precision digits), etc.

126. What does #include <stdarg.h> provide?

A) Standard argument parsing
B) Macros for implementing variadic functions: va_list, va_start, va_arg, va_end
C) Command-line argument handling
D) String argument functions

Correct Answer: B)

Explanation: stdarg.h provides the tools for variadic functions (functions with variable number of arguments): va_list type, va_start(), va_arg(), va_end() macros, and va_copy(). Used to implement printf-like functions.

---

127. The macro BUFSIZ is defined in which header?

A) stdlib.h
B) string.h
C) stdio.h
D) limits.h

Correct Answer: C)

Explanation: BUFSIZ is defined in stdio.h and represents the default I/O buffer size used by setbuf(). Its value is typically 512, 1024, or 4096 depending on the implementation.

---

128. What is EOF and where is it defined?

A) End of function, defined in stdlib.h
B) End of file indicator, typically -1, defined in stdio.h
C) End of format, defined in string.h
D) Error output flag, defined in errno.h

Correct Answer: B)

Explanation: EOF (End Of File) is a macro defined in stdio.h, typically as -1. It's returned by functions like getchar(), fgetc(), and scanf() to indicate end of input or an error condition.

---

129. Which header provides the wchar_t type and wide character functions?

A) stdio.h
B) string.h
C) wchar.h
D) unicode.h

Correct Answer: C)

Explanation: wchar.h provides the wchar_t type, wide character I/O functions (fgetwc, fputwc, fwprintf, fwscanf), and wide string functions (wcscpy, wcslen, etc.) for Unicode/wide character support.

---

130. What does #include <time.h> provide?

A) Timer interrupt handlers
B) Types and functions for time/date: time_t, struct tm, time(), mktime(), strftime()
C) CPU clock speed
D) Real-time OS functions

Correct Answer: B)

Explanation: time.h provides the time_t type, struct tm (broken-down time), and functions: time(), difftime(), mktime(), gmtime(), localtime(), strftime() for working with dates and times.

Section 9: Final Advanced Questions

131. What is the output?

```
#define N 3+2
printf("%d", N*N);
```

A) 25
B) 11
C) 10
D) 13

Correct Answer: B)

*Explanation: N*N expands to 3+2*3+2. Due to operator precedence: 3 + (2*3) + 2 = 3+6+2 = 11. This shows why macros should always use parentheses: #define N (3+2) would give 25.*

---

132. Which is the correct way to conditionally include platform-specific code?

A) if(platform==WINDOWS) { ... }
B) #ifdef _WIN32\n/* Windows code */\n#elif defined(__linux__)\n/* Linux code */\n#endif
C) switch(platform) { case WIN: ... }
D) #if PLATFORM\n#include windows\n#endif

Correct Answer: B)

Explanation: Platform-specific code uses preprocessor conditionals with platform macros: _WIN32 for Windows, __linux__ for Linux, __APPLE__ for macOS. This selects code at compile time, not runtime.

---

133. What does the following macro do?

```
#define SWAP(a,b,T) do{T t=a;a=b;b=t;}while(0)
```

A) Swaps two values without a temp variable
B) Swaps two values of type T using a temp variable, safely wrapped in do...while(0)
C) Defines a template
D) Causes a compilation error because T cannot be a macro parameter

Correct Answer: B)

Explanation: This is a generic swap macro. T is the type, allowing SWAP(x, y, int) to swap integers. The do...while(0) wrapper ensures the macro works correctly in all statement contexts.

---

134. What is the role of __FUNCTION__ or __func__ macro?

A) Defines the main function
B) Provides the name of the current function as a string literal
C) Calls the current function
D) Returns the function's return type

Correct Answer: B)

Explanation: __func__ (C99 standard) and __FUNCTION__ (GCC extension) expand to the name of the enclosing function as a string. Used in debugging, logging, and assertion messages.

---

135. What is the correct output?

```
#define MAX(a,b) ((a)>(b)?(a):(b))
int x=5,y=3;
printf("%d",MAX(x++,y++));
```

A) 5
B) 6

C) Implementation-defined due to double evaluation

D) 3

Correct Answer: C)

Explanation: MAX(x++, y++) expands to ((x++)>(y++)?(x++):(y++)). Since x>y, x++ is evaluated twice, incrementing x to 7. This is undefined behavior due to modifying x twice without a sequence point.

Chapter Summary & Key Points

#include

• Use for system headers, "file" for user-defined headers.
• The preprocessor replaces #include with the file's entire contents.
• Use include guards (#ifndef/#define/#endif) or #pragma once to prevent double inclusion.
• Key headers: stdio.h, stdlib.h, string.h, math.h, ctype.h, assert.h.

#define

• Creates macros (object-like) or function-like macros with parameters.
• Always parenthesize parameters and the whole expression in function-like macros.
• Macros have no type checking — prefer inline functions for complex cases.
• Predefined macros: __LINE__, __FILE__, __DATE__, __TIME__, __STDC__.
• Operators: # (stringify), ## (token-paste).

#ifdef / #ifndef

• #ifdef: block compiled if macro IS defined.
• #ifndef: block compiled if macro is NOT defined.
• Include guards use #ifndef.
• Can be nested; each must be paired with #endif.
• #elif allows multiple conditions; #else provides alternative.
• #if defined(X) allows logical combinations: &&, ||, !.

#undef

• Removes a macro definition.
• Undefining an undefined macro is safe (silently ignored).
• Use #undef then #define to change a macro's value.
• Avoid undefining predefined macros.

#error

• Halts compilation with a custom error message.
• Used to enforce build requirements (platform, version, configuration).
• Evaluated at preprocessing time, not runtime.
• #warning (GCC extension) issues a warning without stopping compilation.

Mock Test 1 — Preprocessor Directives (OAVS TGT Level)

Total Questions: 20	Time Allowed: 30 Minutes	Marks: 1 per question
Instructions: Choose the best answer. No negative marking for practice.		

Q1. Which preprocessor directive is used to include a header file in a C program?

A) #import
B) #include
C) #define
D) #header
Answer: _____

Q2. Which header file must be included to use printf() and scanf() in C?

A) stdlib.h
B) string.h
C) math.h
D) stdio.h
Answer: _____

Q3. What is the difference between #include <file> and #include "file"?

A) No difference
B) <file> is for C++, "file" is for C
C) <file> searches system directories; "file" searches current directory first
D) <file> includes binary; "file" includes text
Answer: _____

Q4. What does the following define? #define MAX 100

A) Declares an integer variable MAX with value 100
B) Creates a macro MAX that is text-replaced with 100 before compilation
C) Allocates memory of size 100
D) Defines a function named MAX
Answer: _____

Q5. What is wrong with this macro: #define SQUARE(x) x*x ?

A) Nothing is wrong
B) Missing semicolon
C) No parentheses around x, causing incorrect expansion for expressions
D) #define cannot define functions
Answer: _____

Q6. #define without a replacement value (e.g., #define DEBUG) is used for:

A) Defining an empty string
B) Conditional compilation with #ifdef/#ifndef
C) This is a syntax error
D) Creating a null pointer
Answer: _____

Q7. What does __DATE__ macro expand to?

A) The current runtime date
B) The compilation date as a string (e.g., "Jan 01 2024")
C) The date the file was last modified
D) A timestamp integer

Answer: _____

Q8. What does #ifdef stand for?

A) if defined — checks if a macro is defined
B) if block defined
C) integer function defined
D) include file defined
Answer: _____

Q9. Which directive is used to end a conditional compilation block?

A) #end
B) #close
C) #endif
D) #stop
Answer: _____

Q10. What does #elif stand for?

A) else include
B) else if — allows chaining conditional compilation checks
C) end link
D) external library
Answer: _____

Q11. What is the result of using #undef on an undefined macro?

A) Compilation error
B) Runtime error
C) No effect (it is silently ignored)
D) Warning and termination
Answer: _____

Q12. Can you #undef a predefined macro like __LINE__?

A) Yes, and it works normally
B) It is undefined behavior
C) No, predefined macros cannot be undefined
D) It causes a compile error in all compilers
Answer: _____

Q13. What is the correct syntax of the #error directive?

A) #error("message")
B) #error: message
C) #error message
D) error #"message"
Answer: _____

Q14. Can #error appear inside a #ifdef block?

A) No
B) Yes — it only triggers if the enclosing condition is true
C) Only in C11
D) Only as the first directive in a file
Answer: _____

Q15. Which of the following is NOT a preprocessor directive?

A) #include
B) #define
C) #while

D) #ifdef

Answer: _____

Q16. What is the translation phase order in C compilation?

A) Compilation → Preprocessing → Linking

B) Preprocessing → Compilation → Assembly → Linking

C) Linking → Compilation → Preprocessing

D) Assembly → Preprocessing → Compilation

Answer: _____

Q17. What does #define TRUE 1 establish?

A) A variable named TRUE equal to 1

B) A macro that replaces TRUE with 1 throughout the code

C) A constant of type bool

D) A function returning 1

Answer: _____

Q18. The # operator in a macro is called:

A) Token pasting

B) Stringification/Stringizing

C) Concatenation

D) Dereferencing

Answer: _____

Q19. Which of the following headers provides INT_MAX and INT_MIN?

A) limits.h

B) stdio.h

C) stdlib.h

D) math.h

Answer: _____

Q20. What is the output?

```
#define CAT(a,b) a##b
int xy = 10;
printf("%d", CAT(x,y));
```

A) Error

B) xy

C) 10

D) 0

Answer: _____

Mock Test 1 — Answer Key & Explanations

Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans
Q1	B	Q2	D	Q3	C	Q4	B	Q5	C
Q6	B	Q7	B	Q8	A	Q9	C	Q10	B
Q11	C	Q12	B	Q13	C	Q14	B	Q15	C
Q16	B	Q17	B	Q18	B	Q19	A	Q20	C

Brief Explanations:

Q1 (B): The #include directive tells the preprocessor to insert the contents of another file into the source code at that point. It is used to include standard or user-defined header files.

Q2 (D): stdio.h (Standard Input Output) provides declarations for printf(), scanf(), FILE, and other I/O functions. It must be included for standard I/O operations.

Q3 (C): Angle brackets instruct the preprocessor to search in predefined system include paths. Double quotes first check the current/project directory, then fall back to system paths.

Q4 (B): Macros defined with #define are simple text substitutions. Every occurrence of MAX in the code will be replaced by 100 by the preprocessor before the compiler sees the code.

Q5 (C): SQUARE(2+3) expands to 2+3*2+3 = 13, not 25. The correct definition should be #define SQUARE(x) ((x)*(x)) with parentheses around each parameter and the whole expression.

Q6 (B): A macro defined without a value simply marks that identifier as defined. This is used with #ifdef to conditionally compile sections of code, common for debug builds.

Q7 (B): __DATE__ is a predefined macro that expands to the compilation date as a string literal in the form "Mmm dd yyyy". It is fixed at compile time, not runtime.

Q8 (A): #ifdef (if defined) is a conditional compilation directive. The code block following #ifdef MACRO is compiled only if MACRO has been defined using #define.

Q9 (C): #endif terminates a conditional compilation block started by #if, #ifdef, or #ifndef. Every conditional directive must be paired with #endif.

Q10 (B): #elif (else if) allows chaining multiple conditions in conditional compilation, similar to else-if in regular C code. It avoids deeply nested #ifdef/#endif structures.

Q11 (C): If you #undef a macro that was never defined or was already undefined, the preprocessor silently ignores it. No error or warning is generated.

Q12 (B): While some compilers may allow it syntactically, undefining predefined macros like __LINE__ or __FILE__ results in undefined behavior according to the C standard. It should be avoided.

Q13 (C): The syntax is #error followed by the error message text (not in quotes). The preprocessor outputs this message and halts compilation.

Q14 (B): #error inside a conditional block only executes if that block's condition is true. This allows conditional error messages, such as: #ifdef WRONG_CONFIG\n#error Bad config\n#endif

Q15 (C): There is no #while preprocessor directive in C. The valid conditional directives are #if, #ifdef, #ifndef, #elif, #else, and #endif. #while is a regular C statement keyword.

Q16 (B): The standard order is: Preprocessing (handle # directives) → Compilation (source to assembly) → Assembly (assembly to object code) → Linking (combine object files into executable).

Q17 (B): This defines a macro. Every occurrence of TRUE in the code after this definition will be replaced by 1 by the preprocessor. No memory is allocated.

Q18 (B): The unary # operator in a macro definition converts a macro parameter to a string literal (surrounded by double quotes). This is called stringification or stringizing.

Q19 (A): limits.h defines macros for the ranges of integer types: INT_MAX, INT_MIN, LONG_MAX, CHAR_MAX, etc. These are implementation-defined constants for the target platform.

Q20 (C): CAT(x,y) uses token-pasting to form the identifier xy. Since int xy = 10 is declared, printf("%d", xy) prints 10.

Mock Test 2 — Preprocessor Directives (OAVS TGT Level)

Total Questions: 20	Time Allowed: 30 Minutes	Marks: 1 per question

Instructions: Choose the best answer. No negative marking for practice.

Q1. What is the correct syntax to include the standard input/output header in C?

A) #include (stdio.h)
B) #include stdio.h
C) #include <stdio.h>
D) include <stdio.h>
Answer: _____

Q2. What happens when the preprocessor encounters #include <stdio.h>?

A) It compiles stdio.h separately
B) It replaces the directive with the content of stdio.h
C) It links the stdio library
D) It creates a copy of stdio.h
Answer: _____

Q3. Which header provides the strcpy() and strlen() functions?

A) stdio.h
B) string.h
C) stdlib.h
D) ctype.h
Answer: _____

Q4. What is the output of: #define SQ(x) x*x

```
printf("%d", SQ(3));
```

A) 9
B) 6
C) 3
D) Error
Answer: _____

Q5. Can macros be defined with multiple lines?

A) No, macros must be on one line
B) Yes, using \ at the end of each line to continue
C) Yes, using { } to enclose lines
D) Yes, but only in C99
Answer: _____

Q6. Which of the following correctly defines a macro for the maximum of two numbers?

A) #define MAX(a,b) a>b?a:b
B) #define MAX(a,b) ((a)>(b)?(a):(b))
C) #define MAX(a,b) = ((a)>(b)?(a):(b))
D) #macro MAX(a,b) ((a)>(b)?(a):(b))
Answer: _____

Q7. Macros are processed at which stage?

A) Runtime
B) Link time
C) Compile time (preprocessing phase)

D) Load time

Answer: _____

Q8. What is the output if DEBUG is not defined?

```
#ifdef DEBUG
printf("Debug mode");
#else
printf("Release");
#endif
```

A) Debug mode

B) Release

C) Nothing

D) Compilation error

Answer: _____

Q9. What is the primary use of #ifndef in header files?

A) To check if a function exists

B) To prevent multiple inclusion of the same header file (include guards)

C) To define new types

D) To import modules

Answer: _____

Q10. Consider: #ifndef PI

```
#define PI 3.14
#endif. What is the purpose?
```

A) Defines PI only if it hasn't been defined already

B) Defines PI always

C) Undefines PI

D) Checks if PI equals 3.14

Answer: _____

Q11. When would you use #undef followed by #define?

A) To change the value of a macro in a specific code section

B) To permanently remove a macro

C) To create a new variable

D) To reset all macros

Answer: _____

Q12. After #undef MAX, what does #ifdef MAX evaluate to?

A) True

B) False — MAX is no longer defined

C) Error

D) True if MAX was defined before

Answer: _____

Q13. When is #error typically used?

A) To debug runtime issues

B) To ensure certain macros are defined or platform requirements are met before compiling

C) To throw C exceptions

D) To print to stderr at runtime

Answer: _____

Q14. Which directive can generate a non-fatal warning during compilation?

A) #error

B) #warning
C) #note
D) #alert
Answer: _____

Q15. Preprocessor directives in C always begin with:

A) //
B) @
C) #
D) $
Answer: _____

Q16. What does #pragma once do?

A) Runs the program once
B) A non-standard but widely supported directive ensuring a header is included only once
C) Defines a one-time macro
D) Compiles a file once per build
Answer: _____

Q17. Which header contains the definition of TRUE and FALSE in C99?

A) stdio.h
B) stdlib.h
C) stdbool.h
D) bool.h
Answer: _____

Q18. Which of the following macro definitions allows computing the minimum of two values safely?

A) #define MIN(a,b) a<b?a:b
B) #define MIN(a,b) ((a)<(b)?(a):(b))
C) #define MIN(a,b) min(a,b)
D) #define MIN = (a<b?a:b)
Answer: _____

Q19. What does #pragma do in C?

A) Defines a standard preprocessor directive
B) Issues implementation-specific instructions to the compiler
C) Includes a binary file
D) Defines a new data type
Answer: _____

Q20. What is macro hygiene in C?

A) Keeping macro names clean and short
B) Using unique names in macro expansions to avoid variable capture
C) Deleting macros after use
D) Writing macros in uppercase only
Answer: _____

Mock Test 2 — Answer Key & Explanations

Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans
Q1	C	Q2	B	Q3	B	Q4	A	Q5	B
Q6	B	Q7	C	Q8	B	Q9	B	Q10	A
Q11	A	Q12	B	Q13	B	Q14	B	Q15	C
Q16	B	Q17	C	Q18	B	Q19	B	Q20	B

Brief Explanations:

Q1 (C): #include <stdio.h> uses angle brackets to include system/standard header files. The angle brackets tell the preprocessor to search in the system include directories.

Q2 (B): The preprocessor literally replaces the #include directive with the entire contents of the specified header file before compilation proceeds.

Q3 (B): string.h provides string handling functions: strcpy(), strcat(), strlen(), strcmp(), strstr(), and many more.

Q4 (A): SQ(3) expands to 3*3 = 9. However, this macro has a common pitfall: SQ(2+1) expands to 2+1*2+1 = 5, not 9. Always use parentheses in macros.

Q5 (B): Macros can span multiple lines using the line-continuation character \. The preprocessor treats all continued lines as a single logical line.

Q6 (B): Option B is correct with full parenthesization. Option A lacks parentheses and can cause wrong results with complex expressions. The keyword is #define, not #macro.

Q7 (C): Macros are processed by the C preprocessor before compilation. The preprocessor performs text substitution and produces modified source code that the compiler then processes.

Q8 (B): Since DEBUG is not defined, #ifdef DEBUG is false. The #else branch executes, printing "Release". This pattern is standard for toggling debug code.

Q9 (B): #ifndef HEADER_H / #define HEADER_H / ... / #endif is the standard include guard pattern. If the header is included again, HEADER_H is already defined so #ifndef is false and the content is skipped.

Q10 (A): This pattern safely defines PI only if it's not already defined. If a user has defined their own PI elsewhere, this won't override it. It's a defensive definition pattern.

Q11 (A): #undef removes the current definition, then #define creates a new one. This allows temporarily redefining a macro with a different value for a specific portion of code.

Q12 (B): After #undef MAX, the macro MAX does not exist. #ifdef MAX checks if MAX is currently defined, which it is not, so it evaluates to false.

Q13 (B): #error is commonly used in conditional compilation: if a required configuration macro is missing, #error forces a meaningful compile-time failure rather than a cryptic compiler error later.

Q14 (B): #warning (a common compiler extension, supported by GCC/Clang) generates a warning message without stopping compilation. Unlike #error, compilation continues after a #warning.

Q15 (C): All preprocessor directives start with the # symbol (hash/pound). They must appear at the beginning of a line (optionally preceded by whitespace in C99 and later).

Q16 (B): #pragma once is a widely used alternative to include guards. It tells the preprocessor to include the file only once per compilation unit. It's not in the C standard but supported by most modern compilers.

Q17 (C): C99 introduced stdbool.h which defines the bool type, true (as 1), and false (as 0) as macros. It also defines __bool_true_false_are_defined.

Q18 (B): Option B uses full parenthesization — both around each parameter and the whole expression — making it safe for complex expressions. Option A can produce incorrect results with operators of lower precedence.

Q19 (B): #pragma provides implementation-defined instructions. Common uses: #pragma once (include guard), #pragma pack (structure packing), #pragma warning (warning control). Unknown pragmas are typically ignored.

Q20 (B): Macro hygiene refers to avoiding unintended interactions between macro-expanded names and names in the surrounding code (variable capture). C macros are not hygienic; this is a known limitation.